ASSIGNMENT REPORT-2

Submitted by :

KARTHIKEYAN M [26120004]

MIRTHULA R [261200121]

Under the guidance of :

HEMA ARUNACHALAM MURTHY

For The Subject

Pattern Recognition and Machine Learning

(26AI5707)

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

SHIV NADAR UNIVERSITY

KALAVAKAM

AUGUST 2026

INTRODUCTION:

This lab is all about dimensionality reduction. We use decomposition techniques such as eigenvalue decomposition (diagonalization) and singular value decomposition to decompose an image and try to reconstruct it with fewer dimensions than the original amount. The square image gets both EVD and SVD, while the rectangular image only gets SVD. Both these techniques do the similar thing: they decompose the matrix (image representation) into its eigenvalues/eigenvectors or singular values/vectors, and then try to reconstruct the original image with as few of these as possible. This makes a compressed, approximate form of the image.

Experiments Performed:

Question 1: Square image (EVD + SVD)

Pipeline.

- **Load:** first we use matplotlib image to load the image.
- **Resize:** then we resize it to 100×100 (it's a square image, so both sides just become 100).
- **Grayscale:** we convert it into grayscale by extracting the RGB channels and using the arithmetic formula $0.299R + 0.587G + 0.114B$.

Then we perform the decomposition and reconstruct using different values of k.

Figure 1: Load $\rightarrow$ resize $\rightarrow$ grayscale, square image (A is 100×100, values in $[0, 255]$).

EVD. Eigenvalue decomposition, also known as diagonalization, is a process used to decompose a square matrix into a diagonal matrix Λ and its eigenbasis:

$$A = Q\Lambda Q^{-1} \tag{1}$$

Here Q contains the eigenbasis (the eigenvectors) and Λ is the diagonal matrix containing the eigenvalues. There are multiple reasons we do this, one of them being to study the matrix through its eigenvalues and eigenvectors.

SVD. While diagonalization is helpful, it can only be performed on square matrices. SVD extends this idea to non-square matrices:

$$A = U\Sigma V^t \tag{2}$$

We have two matrices, AA^T giving us eigenvalues and corresponding eigenvectors stacked in U, and A^TA giving us eigenvalues and corresponding eigenvectors stacked in V. Σ is the singular value matrix, sharing the same dimensions as A, containing the singular values, and singular values are the square root of the eigenvalues. The basic idea is that a non-square matrix cannot be diagonalized directly, so AA^T and A^TA are used to make it square first.

Choice of k. We chose $k = 5, 20, 50$: jumping from a very low point ($k = 5$, heavy compression) up to $k = 50$ (half the image's dimension), with $k = 20$ in between, to show the difference in errors across that range.

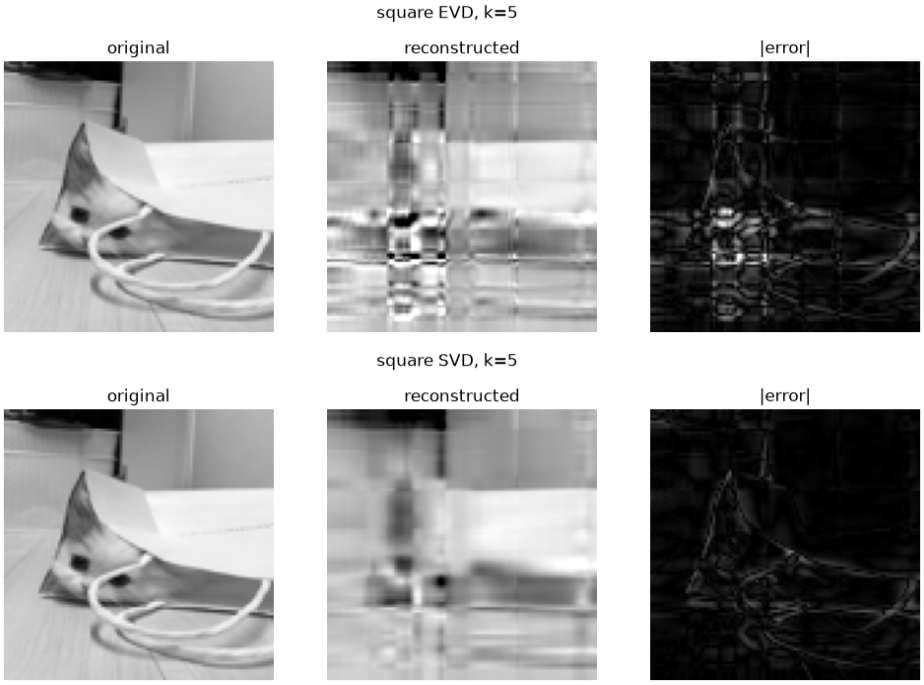

Figure 2: Square image, $k = 5$: EVD (top) vs SVD (bottom).

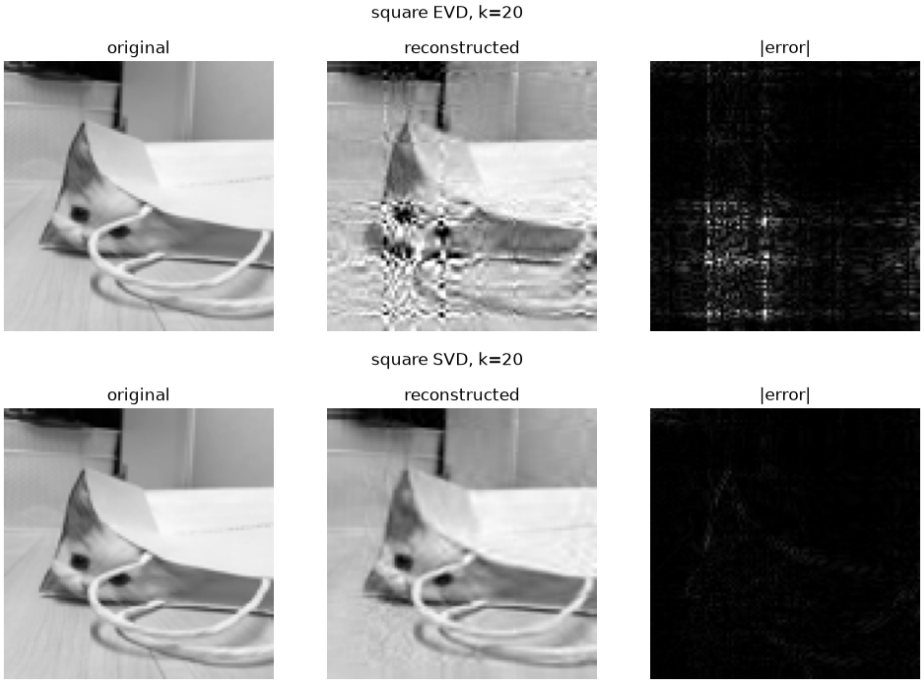

Figure 3: Square image, $k = 20$: EVD (top) vs SVD (bottom).

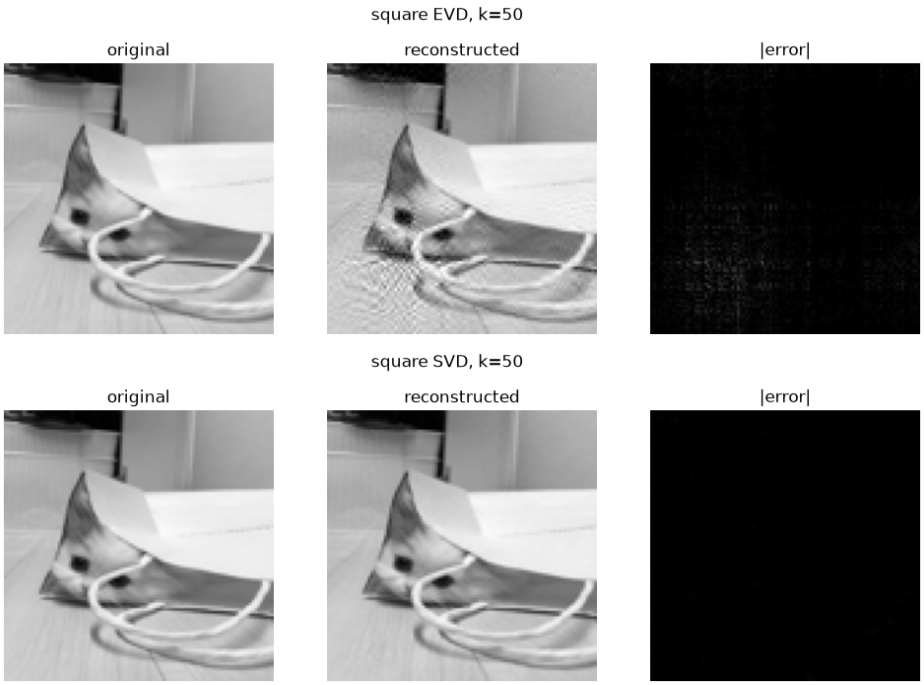

Figure 4: Square image, $k = 50$: EVD (top) vs SVD (bottom).

SVD performs considerably better than EVD, even for this square image. SVD's U and V are orthogonal no matter what A looks like. Q, on the other hand, is only orthogonal

when A itself is symmetric, but it's not in our case, so orthogonality is causing the difference there.

k	$E(k)$ EVD	$E(k)$ SVD	ratio
5	3281.02	1736.93	∼1.9x
20	2575.44	581.84	∼4.4x
50	765.98	122.67	∼6.2x

Table 1: Reconstruction error, square image.

SVD's error is consistently lower than EVD's at every k, and the gap actually widens as k grows (∼1.9x at $k = 5$ up to ∼6.2x at $k = 50$).

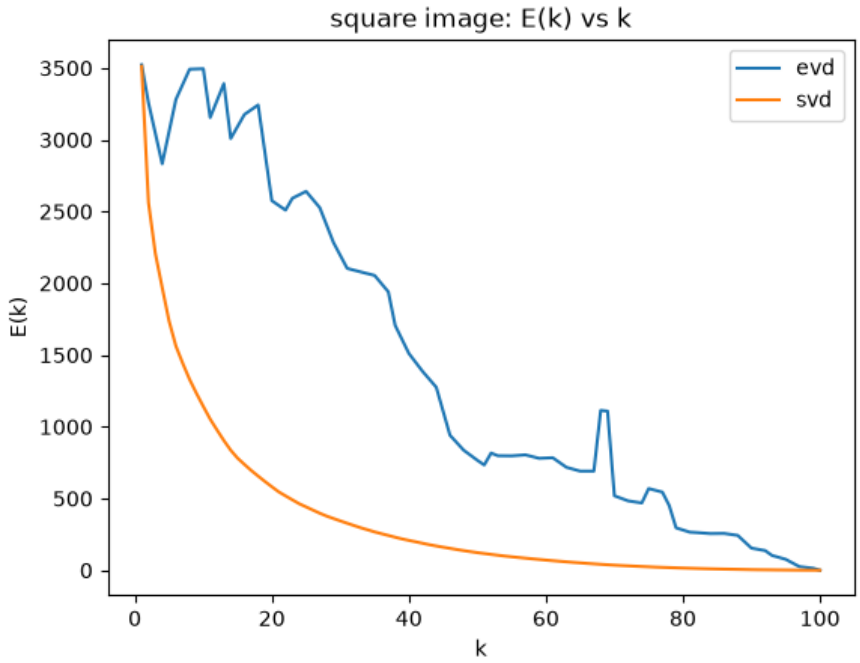

Figure 5: $E(k)$ vs k, square image, all k from 1 to 100.

SVD guarantees orthogonality. The orthogonal vectors are perpendicular to each other, so none of them overlap, and each one is new information on its own. When you keep the important ones and drop the smaller, unwanted ones, the error visibly drops. That is why EVD's error fluctuates a little instead of gradually decreasing, while SVD's error gradually decreases.

Complex eigenvalues. 82 out of 100 eigenvalues came out complex. This is because A isn't symmetric, so the characteristic polynomial's roots aren't guaranteed to be real, and it splits out complex numbers instead. In the code, this is handled by not splitting conjugate pairs across the k boundary when truncating, and then stripping out the imaginary part and keeping only the real part before the result becomes a displayable image.

A Deep Look into EVD at $k = 20$

If we write the EVD as sum of outer products, scaled by corresponding eigen value then. Each EVD layer is $L_i = \lambda_i q_i r_i$, where q_i is a column of Q and r_i is a row of Q^{-1}. Its size is $\|L_i\|_F = |\lambda_i| \|q_i\|_2 \|r_i\|_2$.

i	λ_i	$\lvert\lambda_i\rvert$	$\|q_i\|$	$\|r_i\|$	layer size
0	17299.45	17299.5	1.0	1.0	17685
1	-976.95	977.0	1.0	3.4	3293
2	-476.72 + 198.85i	516.5	1.0	4.1	2138
3	-476.72 - 198.85i	516.5	1.0	4.1	2138
4	241.92 + 316.27i	398.2	1.0	4.7	1886
5	241.92 - 316.27i	398.2	1.0	4.7	1886
6	309.62 + 87.19i	321.7	1.0	10.0	3212
7	309.62 - 87.19i	321.7	1.0	10.0	3212
8	-66.15 + 225.62i	235.1	1.0	5.3	1256
9	-66.15 - 225.62i	235.1	1.0	5.3	1256
10	182.44	182.4	1.0	10.1	1851
11	-171.43 - 58.55i	181.2	1.0	10.3	1858
12	-171.43 + 58.55i	181.2	1.0	10.3	1858
13	-177.14	177.1	1.0	8.5	1503
14	97.03 + 89.13i	131.8	1.0	10.2	1348
15	97.03 - 89.13i	131.8	1.0	10.2	1348
16	32.23 + 117.24i	121.6	1.0	12.4	1513
17	32.23 - 117.24i	121.6	1.0	12.4	1513
18	**115.92 + 3.91i**	**116.0**	**1.0**	**88.8**	**10297**
19	**115.92 - 3.91i**	**116.0**	**1.0**	**88.8**	**10297**
20	-26.14 + 96.61i	100.1	1.0	10.3	1027
21	-26.14 - 96.61i	100.1	1.0	10.3	1027

Table 2: The first 20 EVD layers kept at $k = 20$; indices 20 and 21 are shown only for comparison.

At $k = 20$, both members of the conjugate pair (18 and 19) are kept, so their imaginary parts cancel correctly, keeping only the real part. Their nearly parallel eigenvectors make the corresponding rows of Q^{-1} very large, creating two unusually large layers of size 10,297. Since the other EVD layers are not orthogonal, the whole image reconstruction depends on layers canceling out each other and truncating them distrupts the overall cancellation, that's why the streaks appear.

Question 2: Rectangular image (SVD only)

EVD can only handle square matrices and can't handle non-square matrices. That's the main reason why we have SVD in the first place.

From the image, the width and height are extracted, and then the maximum side is fixed to 100 and the other side is resized to maintain the aspect ratio using (min_side × 100 / max_side).

Figure 6: Load → resize (aspect-preserving) → grayscale, rectangular image ($520 \times 600 \rightarrow 87 \times 100$).

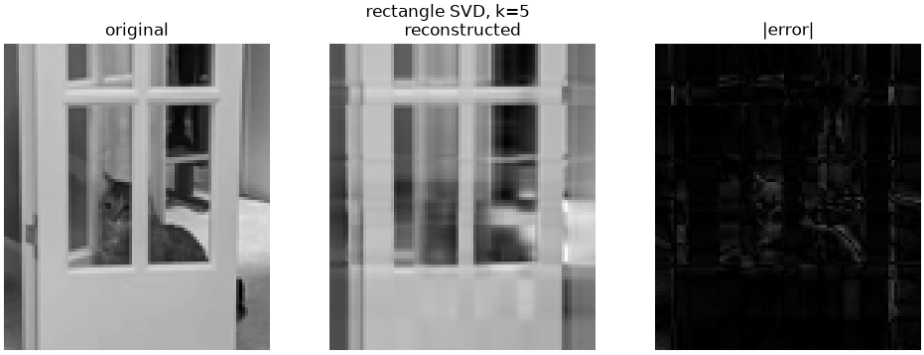

Figure 7: Rectangular image, SVD, $k = 5$.

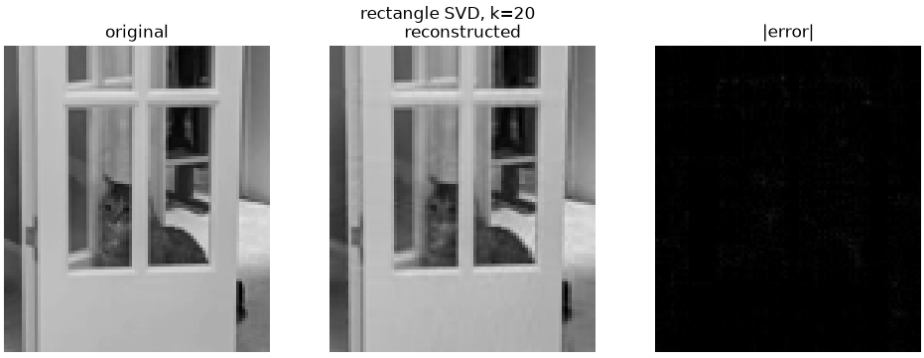

Figure 8: Rectangular image, SVD, $k = 20$.

Figure 9: Rectangular image, SVD, $k = 50$.

k	$E(k)$ SVD
5	1100.14
20	251.75
50	34.15

Table 3: Reconstruction error, rectangular image.

The error drops gradually and smoothly as k increases, the same monotonic behaviour SVD showed on the square image.

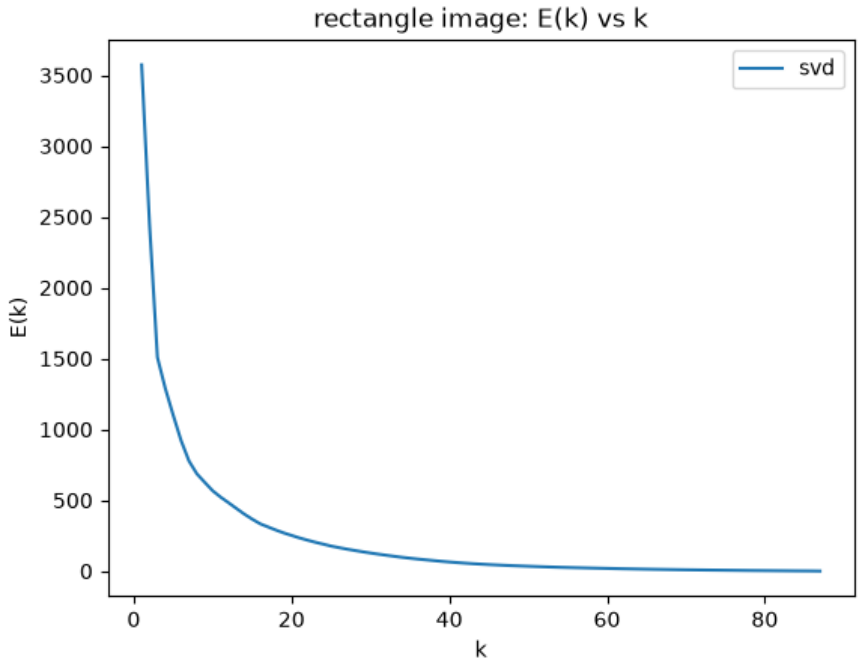

Figure 10: $E(k)$ vs k, rectangular image, all k from 1 to 87.

Inference

If it's a square matrix and symmetric already, just go with EVD because it's less computationally complex than SVD. Reach for SVD whenever there is a need to handle non-square matrices.

Complex eigenvalues really took its time to land. We pondered a lot just to end up getting the justification from the quadratic roots formula.

Another interesting thing we learned while debugging the k=20 case is that, when columns are barely linearly independent, the inverse vectors explode in large numbers.